In [183…

```python
# ==============================================================================
# GROUP PROJECT #17 — MORTGAGE APPROVAL PREDICTION
# GRAD 50400 | Spring 2026
#
#
# MODEL: Random Forest Classifier
# Data: HMDA 2023 & 2024 - OH, WV, IN, KY
# ==============================================================================
import pandas as pd
import numpy as np
import time
import warnings
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import seaborn as sns

from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_score, RandomizedSearchCV
from sklearn.metrics import accuracy_score,precision_score,recall_score,f1_score,confusion_matrix,classification_report
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.pipeline import Pipeline

warnings.filterwarnings("ignore")
```

In [172…

```python
# ==============================================================================
# 1: CONFIGURATION
# ==============================================================================

# Features selected
FEATURES = [
    "loan_amount",
    "loan_to_value_ratio",
    "interest_rate",
    "income",
    "debt_to_income_ratio",
    "applicant_credit_score_type",
    "property_value",
]
```

```python
TARGET = "action_taken"


# action_taken encoding
#   1 = Loan originated OR Application approved, not accepted   --> APPROVED (1)
#   All other codes (denied, withdrawn, incomplete, etc.)       --> DENIED   (0)
APPROVED_CODES = {1, 2}   # originated + approved-not-accepted

# Paths - downloaded HMDA CSV files
PATH_2023 = "state_IN-KY-OH-WV_2023.csv"   # train / out-of-time base
PATH_2024 = "state_IN-KY-OH-WV_2024.csv"   # primary training & holdout

RANDOM_STATE = 42
```

In [173…

```python
# ============================================================================
# 2: DATA LOADING & CLEANING
# ============================================================================

def load_and_clean(path: str, year: int) -> pd.DataFrame:
    """
    Load one HMDA CSV, apply all cleaning steps:
      - Convert non-numeric sentinel values (e.g. 'Exempt') --> NaN
      - Median imputation for numeric columns
      - Factorize categorical column (applicant_credit_score_type)
      - Encode binary target
    """
    print(f"\n{'='*60}")
    print(f"  Loading {year} HMDA data from: {path}")
    print(f"{'='*60}")

    df = pd.read_csv(path, low_memory=False)
    print(f"  Raw shape: {df.shape}")

    # ----------------- Keep only the columns we need -----------------
    cols_needed = FEATURES + [TARGET]
    missing_cols = [c for c in cols_needed if c not in df.columns]
    if missing_cols:
        raise ValueError(f"Missing columns in {year} dataset: {missing_cols}")
    df = df[cols_needed].copy()
```

```python
# ----------------- Binary target encoding -----------------
df[TARGET] = df[TARGET].apply(lambda x: 1 if x in APPROVED_CODES else 0)

# ----------------- Non-numeric sentinel --> NaN -----------------
numeric_cols = [
    "loan_amount", "loan_to_value_ratio", "interest_rate",
    "income", "property_value",
]
for col in numeric_cols:
    df[col] = pd.to_numeric(df[col], errors="coerce")

# ----------------- debt_to_income_ratio often comes as strings like "20%-<30%" — extract midpoint-----------------
def parse_dti(val):
    if pd.isna(val):
        return np.nan
    val = str(val).strip()
    try:
        return float(val)
    except ValueError:
        # handle range strings e.g. "20%-<30%"
        val = val.replace("%", "").replace("<", "").replace(">", "")
        parts = val.split("-")
        nums = []
        for p in parts:
            try:
                nums.append(float(p.strip()))
            except ValueError:
                pass
        return np.mean(nums) if nums else np.nan

df["debt_to_income_ratio"] = df["debt_to_income_ratio"].apply(parse_dti)

# ----------------- Median imputation -----------------
impute_cols = numeric_cols + ["debt_to_income_ratio"]
for col in impute_cols:
    median_val = df[col].median()
    n_missing = df[col].isna().sum()
    if n_missing > 0:
        print(f"  Imputing {n_missing:,} missing values in '{col}' "
              f"with median={median_val:.4f}")
    df[col].fillna(median_val, inplace=True)
```

```python
    # ----------------- Factorize categorical -----------------
    df["applicant_credit_score_type"], _ = pd.factorize(
        df["applicant_credit_score_type"].astype(str)
    )

    # ----------------- Drop any rows still containing NaN -----------------
    before = len(df)
    df.dropna(inplace=True)
    after = len(df)
    if before != after:
        print(f"  Dropped {before - after:,} rows with remaining NaNs.")


    # ----------------- Class balance review -----------------
    counts = df[TARGET].value_counts()
    pct_approved = counts.get(1, 0) / len(df) * 100
    pct_denied   = counts.get(0, 0) / len(df) * 100
    print(f"\n  Class Balance ({year}):")
    print(f"    Approved (1): {counts.get(1,0):>10,}  ({pct_approved:.1f}%)")
    print(f"    Denied   (0): {counts.get(0,0):>10,}  ({pct_denied:.1f}%)")
    print(f"  Final shape:  {df.shape}")

    return df
```

In [174…

```python
# ============================================================================
# 3: SCALABILITY TEST (Proposal 3)
# ============================================================================

# For Random Forest
def scalability_test_rf(X_train, y_train, X_test, y_test):
    """
    Train RF at 10 / 25 / 50 / 100 % of training data and record
    training + inference runtimes:
      - Inference  <= 1s per 10 k records
      - Training   <= 10 min
    """
    print("\n" + "="*60)
    print("  SCALABILITY TEST [Random Forest]")
    print("="*60)
```

```python
    fractions = [0.10, 0.25, 0.50, 1.00]
    results = []

    for frac in fractions:
        n = int(len(X_train) * frac)
        X_sub = X_train.iloc[:n]
        y_sub = y_train.iloc[:n]

        rf_quick = RandomForestClassifier(
            n_estimators=50, max_depth=10,
            random_state=RANDOM_STATE, n_jobs=-1
        )

        t0 = time.time()
        rf_quick.fit(X_sub, y_sub)
        train_time = time.time() - t0

        t0 = time.time()
        rf_quick.predict(X_test)
        infer_time = time.time() - t0
        infer_per_10k = (infer_time / len(X_test)) * 10_000

        acc = accuracy_score(y_test, rf_quick.predict(X_test))

        results.append({
            "Sample Size": f"{frac*100:.0f}%",
            "N Train": n,
            "Train Time (s)": round(train_time, 2),
            "Infer / 10k (s)": round(infer_per_10k, 4),
            "Accuracy": round(acc, 4),
        })
        print(f"RF  {frac*100:3.0f}% ({n:>8,} rows) | "
              f"train={train_time:6.2f}s | "
              f"infer/10k={infer_per_10k:.4f}s | "
              f"acc={acc:.4f}")

    return pd.DataFrame(results)

# For Logistic Regression
def scalability_test_logreg_minmax(X_train, y_train, X_test, y_test):
```

```python
print("\n" + "="*60)
print(" SCALABILITY TEST [Logistic Regression + MinMaxScaler(0,1)]")
print("="*60)

fractions = [0.10, 0.25, 0.50, 1.00]
results = []

for frac in fractions:
    n = int(len(X_train) * frac)
    X_sub = X_train.iloc[:n]
    y_sub = y_train.iloc[:n]

    pipe = Pipeline(steps=[
        ("scaler", MinMaxScaler(feature_range=(0, 1))),
        ("lr", LogisticRegression(max_iter=1000, random_state=42, n_jobs=-1))
    ])

    t0 = time.time()
    pipe.fit(X_sub, y_sub)
    train_time = time.time() - t0

    t0 = time.time()
    y_pred = pipe.predict(X_test)
    infer_time = time.time() - t0

    infer_per_10k = (infer_time / len(X_test)) * 10_000
    acc = accuracy_score(y_test, y_pred)

    results.append({
        "Sample Size": f"{frac*100:.0f}%",
        "N Train": n,
        "Train Time (s)": round(train_time, 2),
        "Infer / 10k (s)": round(infer_per_10k, 4),
        "Accuracy": round(acc, 4),
    })

    print(
        f"LR+MM {frac*100:3.0f}% ({n:>8,} rows) | "
        f"train={train_time:6.2f}s | infer/10k={infer_per_10k:.4f}s | acc={acc:.4f}"
    )
```

```python
    return pd.DataFrame(results)
```

In [175…

```python
# ============================================================================
# 4: HYPERPARAMETER TUNING
# ============================================================================

# For Random Forest
def tune_random_forest_fast(X_train, y_train):
    """
    RandomizedSearchCV over the hyperparameter grid:
      - n_estimators
      - max_depth
      - min_samples_split / min_samples_leaf
      - class_weight  (handles class imbalance)
    """
    print("\n" + "="*60)
    print("  HYPERPARAMETER TUNING - [Random Forest] - RandomizedSearchCV (3-fold CV)")
    print("="*60)

    # It took overmore than 9+ hrs for full dataset, so making below changes
    #Reduce n_iter (number of random combinations) from 30 to 10
    #Use a smaller subset of the training data for tuning (e.g., 10–20%)
    #Limit the parameter grid to fewer
    #Use n_jobs=-1 to parallelize
    #Reduce CV Folds: 5 CV--> 3 CV # saves ~40% of compute vs cv=5, still statistically meaningful

    frac = 0.2  # 20% of training data
    n = int(len(X_train) * frac)
    X_sub = X_train.iloc[:n]
    y_sub = y_train.iloc[:n]
    param_dist = {
        "n_estimators": [100, 200],         # How many trees to build in the forest # droppedd 300,500
        "max_depth": [10, 20, None],        # How deep each tree is allowed to grow # dropped 30-redundant with None
        "min_samples_split": [5, 10],       # Minimum data points needed to split a node # dropped 2
        "min_samples_leaf": [1, 2],         # Minimum data points required at a leaf node # dropped 4
        "max_features": ["sqrt"],           # How many features each tree considers at each split
        "class_weight": [None, "balanced"], # How much to penalize mistakes on minority vs majority class
    }
    rf_base = RandomForestClassifier(random_state=42, n_jobs=-1)
```

```python
    search = RandomizedSearchCV(
        estimator=rf_base,
        param_distributions=param_dist,
        n_iter=10,  # Reduced from 30
        cv=3,       # Reduced from 5 to 3 as it was taking too long
        scoring="f1",
        random_state=42,
        n_jobs=-1,
        verbose=1,
    )

    t0 = time.time()
    search.fit(X_train, y_train)

    print(f"\nRF tuning completed in {time.time() - t0:.1f}s")
    print(f"RF Best CV F1: {search.best_score_:.4f}")
    print(f"RF Best params: {search.best_params_}")

    return search.best_estimator_, search.best_params_

# For Logistic Regression
def tune_logistic_regression_minmax_verbose(X_train, y_train):
    """
    Logistic Regression with MinMax scaling to [0,1] via Pipeline.
    Returns: (best_estimator_pipeline, best_params, cv_results_df)
    """
    print("\n" + "="*60)
    print("  HYPERPARAMETER TUNING - [LogReg + MinMaxScaler(0,1)] - GridSearchCV (3-fold CV)")
    print("="*60)

    pipe = Pipeline(steps=[
        ("scaler", MinMaxScaler(feature_range=(0, 1))),
        ("lr", LogisticRegression(random_state=RANDOM_STATE, n_jobs=-1))
    ])

    # Expanded grid to better support "more time tuning"
    param_grid = {
        "lr__C": [0.001, 0.01, 0.1, 1, 10, 100],
        "lr__class_weight": [None, "balanced"],
        "lr__solver": ["lbfgs", "liblinear"],
```

```
        "lr__penalty": ["l2"],
        "lr__max_iter": [1000, 3000]
    }

    search = GridSearchCV(
        estimator=pipe,
        param_grid=param_grid,
        cv=3,
        scoring="f1",
        n_jobs=-1,
        verbose=1,
        return_train_score=True
    )

    t0 = time.time()
    search.fit(X_train, y_train)
    print(f"\n  LogReg tuning completed in {time.time() - t0:.1f}s")
    print(f"  LogReg Best CV F1   : {search.best_score_:.4f}")
    print(f"  LogReg Best params  : {search.best_params_}")

    # Table of attempts (use in write-up to discuss impact)
    results = pd.DataFrame(search.cv_results_)
    keep = [
        "rank_test_score",
        "mean_test_score", "std_test_score",
        "mean_train_score", "std_train_score",
        "params"
    ]
    results = results[keep].sort_values(["rank_test_score", "mean_test_score"], ascending=[True, False])

    print("\n  Top 15 hyperparameter attempts (by rank):")
    print(results.head(15).to_string(index=False))

    return search.best_estimator_, search.best_params_, results
```

In [176…

```
# ============================================================================
# 5: EVALUATION HELPERS
# ============================================================================

def evaluate_model(model, X_test, y_test, scaler=None, label=""):
```

```python
    """
    Compute all metrics from the evaluation plan: accuracy, precision, recall, F1, confusion matrix
    """
    if scaler:
        X_test = scaler.transform(X_test)

    y_pred = model.predict(X_test)

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, zero_division=0)
    rec = recall_score(y_test, y_pred, zero_division=0)
    f1 = f1_score(y_test, y_pred, zero_division=0)
    cm = confusion_matrix(y_test, y_pred)

    print(f"\n{'='*60}")
    print(f" EVALUATION RESULTS {label}")
    print(f"{'='*60}")
    print(f" Accuracy : {acc:.4f}")
    print(f" Precision : {prec:.4f}")
    print(f" Recall : {rec:.4f}")
    print(f" F1-Score : {f1:.4f}")
    print(f"\n Confusion Matrix: {label}\n{cm}")
    print(f"\n Classification Report: {label}\n{classification_report(y_test, y_pred, target_names=['Denied','Approved'])}")

    # Success criteria check
    print(f"  SUCCESS CRITERIA CHECK: {label}")
    print(f"    Accuracy >= 0.90  --> {'PASS' if acc  >= 0.90 else 'FAIL'} ({acc:.4f})")
    print(f"    Precision >= 0.90 --> {'PASS' if prec >= 0.90 else 'FAIL'} ({prec:.4f})")
    print(f"    Recall >= 0.90    --> {'PASS' if rec  >= 0.90 else 'FAIL'} ({rec:.4f})")
    print(f"    F1 >= 0.90        --> {'PASS' if f1   >= 0.90 else 'FAIL'} ({f1:.4f})")

    return {"accuracy": acc, "precision": prec, "recall": rec, "f1": f1, "cm": cm, "y_pred": y_pred}
```

In [177…

```python
# ========================
# 6. LOGISTIC REGRESSION - COEFFICICIENT/ODDS-RATIO INTERPRETATION
# ========================

import numpy as np
import pandas as pd
from sklearn.pipeline import Pipeline
```

```python
def interpret_logistic_regression(model, feature_names):
    """
    Works with:
      - LogisticRegression (has .coef_)
      - Pipeline(..., ('lr', LogisticRegression())) (coef at named_steps['lr'])
    Returns a DataFrame with coefficients and odds ratios.
    """
    # If Pipeline, pull out the LR step
    if isinstance(model, Pipeline):
        if "lr" not in model.named_steps:
            raise ValueError("Pipeline model does not contain a step named 'lr'.")
        lr = model.named_steps["lr"]
    else:
        lr = model

    if not hasattr(lr, "coef_"):
        raise AttributeError("Provided model does not have coef_. Fit the model before interpreting.")

    coefs = lr.coef_[0]
    odds_ratios = np.exp(coefs)

    importance_df = pd.DataFrame({
        "Feature": feature_names,
        "Coefficient": coefs,
        "Odds Ratio": odds_ratios,
        "Abs(Coefficient)": np.abs(coefs)
    }).sort_values(by="Abs(Coefficient)", ascending=False)

    print("\n[Logistic Regression] - coefficient/odds-ratio interpretation (sorted by |coefficient|):")
    print(importance_df[["Feature", "Coefficient", "Odds Ratio"]])

    return importance_df
```

In [178…

```python
# ========================
# 7. VISUALIZATION
# ========================

def plot_all_comparison(rf_model, lr_model, X_test, y_test, rf_metrics, lr_metrics, rf_feature_names,
                        lr_feature_names, scale_df_rf, scale_df_lr, out_label="rf_lr_results"):
```

```python
fig, axs = plt.subplots(2, 3, figsize=(22, 12))
fig.suptitle("\nGroup 17 — Random Forest: Mortgage Approval Prediction\n"
    "HMDA 2024 | OH, WV, IN, KY", fontsize=18, fontweight="bold", y=0.98)

# 1. Confusion Matrix RF
from sklearn.metrics import ConfusionMatrixDisplay
ConfusionMatrixDisplay(rf_metrics["cm"], display_labels=["Denied (0)", "Approved (1)"]).plot(
    ax=axs[0,0], colorbar=False, cmap="Blues")
axs[0,0].set_title("Random Forest\nConfusion Matrix", fontsize=13, fontweight="bold")
axs[0,0].set_xlabel("Predicted", fontsize=11)
axs[0,0].set_ylabel("Actual", fontsize=11)

# 2. Confusion Matrix LR
ConfusionMatrixDisplay(lr_metrics["cm"], display_labels=["Denied (0)", "Approved (1)"]).plot(
    ax=axs[0,1], colorbar=False, cmap="Greens")
axs[0,1].set_title("Logistic Regression\nConfusion Matrix", fontsize=13, fontweight="bold")
axs[0,1].set_xlabel("Predicted", fontsize=11)
axs[0,1].set_ylabel("Actual", fontsize=11)

# 3. Feature Importance RF
importances = rf_model.feature_importances_
indices = np.argsort(importances)[::-1]
sorted_features = [rf_feature_names[i] for i in indices]
sorted_imp = importances[indices]
# Build a results table (features + importance)
rf_importance_df = pd.DataFrame({"Feature": sorted_features,"Importance Score": sorted_imp})
print("\n[Random Forest] - Feature Importance :")
print(rf_importance_df)
# Plot
axs[0,2].barh(sorted_features[::-1], sorted_imp[::-1],
              color=sns.color_palette("Blues_r", len(sorted_features)))
axs[0,2].set_title("Random Forest\nFeature Importance", fontsize=13, fontweight="bold")
axs[0,2].set_xlabel("Importance Score", fontsize=11)
axs[0,2].set_ylabel("Features", fontsize=11)

# 4. Feature Importance LR (Odds Ratio)
# Also builds a results table (features - Coefficients/oddsratio)
lr_importance_df = interpret_logistic_regression(lr_model, lr_feature_names)
# Plot
axs[1,0].barh(lr_importance_df["Feature"][::-1], lr_importance_df["Odds Ratio"][::-1],
              color=sns.color_palette("Greens_r", len(lr_importance_df)))
```

```python
axs[1,0].set_title("Logistic Regression\nOdds Ratios", fontsize=13, fontweight="bold")
axs[1,0].set_xlabel("Odds Ratio", fontsize=11)
axs[0,2].set_ylabel("Features", fontsize=11)

# 5. Metrics Summary
metric_names = ["Accuracy", "Precision", "Recall", "F1-Score"]
rf_vals = [rf_metrics["accuracy"], rf_metrics["precision"], rf_metrics["recall"], rf_metrics["f1"]]
lr_vals = [lr_metrics["accuracy"], lr_metrics["precision"], lr_metrics["recall"], lr_metrics["f1"]]
x = np.arange(len(metric_names))
width = 0.35
axs[1,1].bar(x - width/2, rf_vals, width, label='Random Forest', color="#2196F3")
axs[1,1].bar(x + width/2, lr_vals, width, label='Logistic Regression', color="#43A047")
# Addedd threshold success
axs[1,1].axhline(y=0.90, color="green", linestyle="--", linewidth=1.5, label="Success threshold (0.90)")
axs[1,1].set_xticks(x)
axs[1,1].set_xticklabels(metric_names)
axs[1,1].set_ylim(0, 1.10)
axs[1,1].set_xlabel("Metrics Name", fontsize=11)
axs[1,1].set_ylabel("Score", fontsize=11)
axs[1,1].set_title("Performance Metrics", fontsize=13, fontweight="bold")
axs[1,1].legend()

# 6. Scalability: Training Time (Bar Chart) + Accuracy (Line) Side-by-Side
ax = axs[1,2]  # or use your own axes reference

if (scale_df_rf is not None and not scale_df_rf.empty and
    scale_df_lr is not None and not scale_df_lr.empty):

    x_labels = scale_df_rf["Sample Size"].tolist()
    x_pos = np.arange(len(x_labels))
    width = 0.35

    # Bar charts for train time
    bars_rf = ax.bar(x_pos - width/2, scale_df_rf["Train Time (s)"], width=width,
                     color="#42A5F5", label="RF Train Time (s)")
    bars_lr = ax.bar(x_pos + width/2, scale_df_lr["Train Time (s)"], width=width,
                     color="#43A047", label="LR Train Time (s)")

    ax.set_xticks(x_pos)
    ax.set_xticklabels(x_labels)
    ax.set_xlabel("Sample Size %", fontsize=11)
```

```python
        ax.set_ylabel("Train Time (s)", fontsize=11)
        ax.set_title("Scalability: Training Time vs Sample Size", fontsize=13, fontweight="bold")

        # Accuracy line (shared x-axis, secondary y-axis)
        ax2 = ax.twinx()
        ax2.plot(x_pos, scale_df_rf["Accuracy"], color="darkorange", marker="o", linewidth=2, label="RF Accuracy")
        ax2.plot(x_pos, scale_df_lr["Accuracy"], color="red", marker="s", linewidth=2, label="LR Accuracy")
        ax2.set_ylabel("Accuracy", fontsize=11)
        ax2.tick_params(axis="y", labelcolor="darkorange")
        ax2.set_ylim(0.5, 1.05)

        # Legends
        ax.legend(loc="upper left", fontsize=9)
        ax2.legend(loc="lower right", fontsize=9)

    else:
        ax.text(0.5, 0.5, "Scalability data not available",
                ha="center", va="center", transform=ax.transAxes)

    fig.tight_layout(rect=[0, 0.1, 1, 0.96])
    fig.savefig(f"{out_label}.png", dpi=150, bbox_inches="tight")
    print(f"\n Visualisation saved --> {out_label}.png")
    plt.show()     # <-- This displays the plot inline in the notebook
    plt.close(fig)
```

In [179…

```python
# ========================
# 8. CROSS-VALIDATION
# ========================

def run_cross_validation(model, X_train, y_train, scaler=None, label=""):
    """5-fold CV on training data to confirm generalisation."""
    if scaler:
        X_train = scaler.transform(X_train)
    print("\n" + "="*60)
    print(f" 5-FOLD CROSS-VALIDATION {label}")
    print("="*60)
    for metric in ["accuracy", "f1", "precision", "recall"]:
        scores = cross_val_score(model, X_train, y_train, cv=5, scoring=metric, n_jobs=-1)
        print(f" {metric:<12}: {scores.mean():.4f} +/- {scores.std():.4f}")
```

In [180…

```python
# ========================
# 9. HELPER FUNCTIONS
# ========================

def print_split_vs_oot_table(rf_split, lr_split, rf_oot, lr_oot):
    df = pd.DataFrame([
        {
            "Model": "Random Forest",
            "80/20 Accuracy": rf_split["accuracy"], "80/20 Precision": rf_split["precision"],
            "80/20 Recall": rf_split["recall"], "80/20 F1": rf_split["f1"],
            "2023->2024 Accuracy": rf_oot["accuracy"], "2023->2024 Precision": rf_oot["precision"],
            "2023->2024 Recall": rf_oot["recall"], "2023->2024 F1": rf_oot["f1"],
        },
        {
            "Model": "Logistic Regression",
            "80/20 Accuracy": lr_split["accuracy"], "80/20 Precision": lr_split["precision"],
            "80/20 Recall": lr_split["recall"], "80/20 F1": lr_split["f1"],
            "2023->2024 Accuracy": lr_oot["accuracy"], "2023->2024 Precision": lr_oot["precision"],
            "2023->2024 Recall": lr_oot["recall"], "2023->2024 F1": lr_oot["f1"],
        }
    ])

    print("\n" + "="*60)
    print("  METRICS COMPARISON: 80/20 HOLDOUT vs OUT-OF-TIME (Train=2023, Test=2024)")
    print("="*60)
    print(df.to_string(index=False))

def plot_predicted_vs_actual_scatter(model, X_test, y_test, title="", use_proba=True, jitter=0.03):
    """
    Scatter: Actual (0/1) vs Predicted.
    - If use_proba=True and model supports predict_proba, plots predicted probability of class 1.
    - Otherwise plots predicted class (0/1).
    """
    y_true = np.asarray(y_test)
    # Predicted values
    if use_proba and hasattr(model, "predict_proba"):
        y_pred_val = model.predict_proba(X_test)[:, 1]  # probability of Approved(1)
        y_label = "Predicted P(Approved=1)"
    else:
        y_pred_val = model.predict(X_test)
```

```
        y_label = "Predicted Class"

    # Jitter actual labels for visibility (since actual is binary 0/1)
    rng = np.random.default_rng(42)
    y_true_j = y_true + rng.normal(0, jitter, size=len(y_true))

    plt.figure(figsize=(8, 5))
    plt.scatter(y_true_j, y_pred_val, s=6, alpha=0.15)
    plt.xticks([0, 1], ["Denied (0)", "Approved (1)"])
    plt.xlabel("Actual")
    plt.ylabel(y_label)
    plt.title(title)

    # Reference lines
    plt.axvline(0, color="gray", linewidth=1, alpha=0.4)
    plt.axvline(1, color="gray", linewidth=1, alpha=0.4)
    if y_label == "Predicted Class":
        plt.axhline(0, color="gray", linewidth=1, alpha=0.4)
        plt.axhline(1, color="gray", linewidth=1, alpha=0.4)

    plt.tight_layout()
    plt.show()
```

In [181…

```
# ========================
# 10. MAIN PIPELINE
# ========================
def main():
    print("\n" + "="*60)
    print("  GROUP PROJECT #17 - MORTGAGE APPROVAL PREDICTION")
    print("  GRAD 50400 | Spring 2026")
    print("="*60)

    # Load & clean 2024 (primary dataset)
    df_2024 = load_and_clean(PATH_2024, 2024)

    X = df_2024[FEATURES]
    y = df_2024[TARGET]

    # 80/20 holdout split
    X_train, X_test, y_train, y_test = train_test_split(
```

```python
    X, y, test_size=0.20, random_state=RANDOM_STATE, stratify=y
)
print(f"\n  Train set: {X_train.shape[0]:,} rows")
print(f"  Test  set: {X_test.shape[0]:,} rows")

# Scalability test (Proposal 3)
scale_df_rf = scalability_test_rf(X_train, y_train, X_test, y_test)
print("\n  Scalability Summary [Random Forest] - (80/20 holdout split) :")
print(scale_df_rf.to_string(index=False))
print("")

scale_df_logreg = scalability_test_logreg(X_train, y_train, X_test, y_test)
print("\n  Scalability Summary [Logistic Regression] - (80/20 holdout split) :")
print(scale_df_logreg.to_string(index=False))

# --- Random Forest (with hyperparameter tuning) ---
best_rf, best_params = tune_random_forest_fast(X_train, y_train)

# --- Evaluate on 2024 holdout test set (Random Forest)
rf_metrics = evaluate_model(best_rf, X_test, y_test, label="[Random Forest] - Evaluate on 2024 holdout test set")
plot_predicted_vs_actual_scatter(    best_rf, X_test, y_test,
title="Random Forest — Predicted vs Actual (Holdout 2024)",    use_proba=True)
t0 = time.time()
infer_time_2024 = time.time() - t0
infer_per_10k_2024 = (infer_time_2024 / len(X_test)) * 10_000
print(f"\n[Random Forest] - Inference time per 10k records (Evaluate on 2024 holdout test set): {infer_per_10k_2024:.4f}s
      f"(target ≤ 1.0s)")


scale_df_rf = scalability_test_rf(X_train, y_train, X_test, y_test)
# 5-fold CV on training data
run_cross_validation(best_rf, X_train, y_train, label="[Random Forest]")
rf_feature_names = FEATURES

# --- Logistic Regression (MinMax [0,1] + more tuning + verbose grid results) ---
best_lr_pipe, best_lr_params, lr_cv_results = tune_logistic_regression_minmax_verbose(X_train, y_train)

# --- Evaluate on 2024 holdout test set (Logistic Regression) ---
lr_metrics = evaluate_model(best_lr_pipe, X_test, y_test,
                            label="[Logistic Regression + MinMax] - Evaluate on 2024 holdout test set")
plot_predicted_vs_actual_scatter(best_lr_pipe, X_test, y_test,
```

```
    title="Logistic Regression + MinMax — Predicted vs Actual (Holdout 2024)",
    use_proba=True)
    t0 = time.time()
    _ = best_lr_pipe.predict(X_test)
    infer_time_2024 = time.time() - t0
    infer_per_10k_2024 = (infer_time_2024 / len(X_test)) * 10_000
    print(f"\n[Logistic Regression + MinMax] - Inference time per 10k records (Evaluate on 2024 holdout test set): "
          f"{infer_per_10k_2024:.4f}s (target ≤ 1.0s)")

    # Scalability test for LR (keep your existing function if you want, but note:
    # your scalability_test_logreg uses StandardScaler internally, not MinMax.
    # If you want consistency, either update scalability_test_logreg or skip it here.
    scale_df_lr = scalability_test_logreg_minmax(X_train, y_train, X_test, y_test)

    # 5-fold CV on training data (Pipeline handles scaling internally)
    run_cross_validation(best_lr_pipe, X_train, y_train, scaler=None, label="[Logistic Regression + MinMax]")

    lr_feature_names = FEATURES


    # --- Out-of-time validation: train on 2023, test on 2024 (Random Forest)
    print("\n" + "="*60)
    print("  OUT-OF-TIME VALIDATION (Train=2023, Test=2024) - [Random Forest]")
    print("="*60)
    rf_oot_metrics = None
    try:
        df_2023 = load_and_clean(PATH_2023, 2023)
        X_2023 = df_2023[FEATURES]
        y_2023 = df_2023[TARGET]

        rf_oot = RandomForestClassifier(**best_params,random_state=RANDOM_STATE, n_jobs=-1)
        rf_oot.fit(X_2023, y_2023)

        # Test on fimportance dataset
        rf_oot_metrics = evaluate_model(rf_oot, X, y, label="Out-of-Time (Train=2023, Test=2024) - [Random Forest]")
    except FileNotFoundError:
        print(f"  NOTE: '{PATH_2023}' not found — skipping out-of-time test.")
        print("  Update PATH_2023 at the top of this file to enable this check.")

    # --- Out-of-time validation: train on 2023, test on 2024 (Logistic Regression + MinMax) ---
    print("\n" + "="*60)
```

```python
print("  OUT-OF-TIME VALIDATION (Train=2023, Test=2024) - [Logistic Regression + MinMax]")
print("="*60)

lr_oot_metrics = None
try:
    df_2023 = load_and_clean(PATH_2023, 2023)
    X_2023 = df_2023[FEATURES]
    y_2023 = df_2023[TARGET]

    # Refit the best pipeline on 2023, then evaluate on 2024
    lr_oot_pipe = best_lr_pipe
    lr_oot_pipe.fit(X_2023, y_2023)

    lr_oot_metrics = evaluate_model(lr_oot_pipe, X, y,
        label="[Out-of-Time (Train=2023, Test=2024) - [Logistic Regression + MinMax]"
    )
except FileNotFoundError:
    print(f"  NOTE: '{PATH_2023}' not found - skipping out-of-time test.")
    print("  Update PATH_2023 at the top of this file to enable this check.")


# --- Compare metrics: 80/20 vs OOT (as requested by peer review) ---
if ("rf_oot_metrics" in locals() and rf_oot_metrics is not None and
    "lr_oot_metrics" in locals() and lr_oot_metrics is not None):
    print_split_vs_oot_table(rf_metrics, lr_metrics, rf_oot_metrics, lr_oot_metrics)
else:
    print("\n  Skipping split-vs-OOT comparison table (OOT metrics missing).")


  # --- Visualizations ---
plot_all_comparison(
    rf_model=best_rf,
    lr_model=best_lr_pipe,
    X_test=X_test,
    y_test=y_test,
    rf_metrics=rf_metrics,
    lr_metrics=lr_metrics,
    rf_feature_names=rf_feature_names,
    lr_feature_names=lr_feature_names,
    scale_df_rf=scale_df_rf,
    scale_df_lr=scale_df_lr,
```

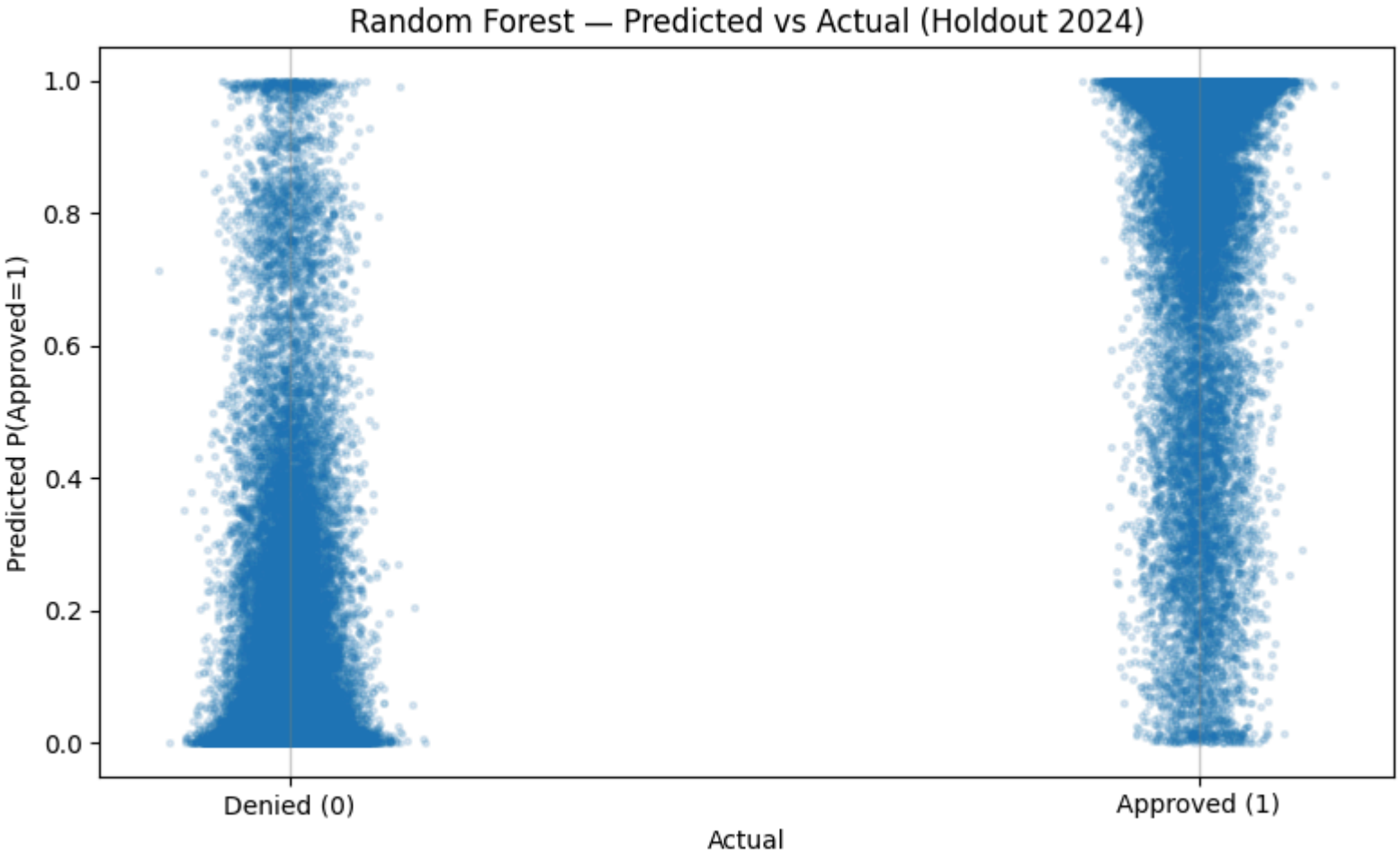
```
        out_label="rf_lr_results"
    )

        # --- Save tuned model summary (Random Forest) ----------------
    print("\n" + "="*60)
    print("  FINAL MODEL SUMMARY [RANDOM FOREST]")
    print("="*60)
    print(f"  Model          : RandomForestClassifier (scikit-learn)")
    print(f"  Best params    : {best_params}")
    print(f"  Features used  : {FEATURES}")
    print(f"  Train rows     : {X_train.shape[0]:,}")
    print(f"  Test rows      : {X_test.shape[0]:,}")
    print(f"  Accuracy       : {rf_metrics['accuracy']:.4f}")
    print(f"  Precision      : {rf_metrics['precision']:.4f}")
    print(f"  Recall         : {rf_metrics['recall']:.4f}")
    print(f"  F1-Score       : {rf_metrics['f1']:.4f}")
    print(f"\n  Visualisation  : group17_rf_results.png")
    print(f"\n  Done.")


    # --- Save tuned model summary (Logistic Regression) ----------------
    print("\n" + "="*60)
    print("  FINAL MODEL SUMMARY [LOGISTIC REGRESSION]")
    print("="*60)
    print(f"  Model          : LogisticRegression (scikit-learn)")
    print(f"  Best params    : {best_lr_params}")
    print(f"  Features used  : {FEATURES}")
    print(f"  Train rows     : {X_train.shape[0]:,}")
    print(f"  Test rows      : {X_test.shape[0]:,}")
    print(f"  Accuracy       : {lr_metrics['accuracy']:.4f}")
    print(f"  Precision      : {lr_metrics['precision']:.4f}")
    print(f"  Recall         : {lr_metrics['recall']:.4f}")
    print(f"  F1-Score       : {lr_metrics['f1']:.4f}")
    print(f"\n  Visualisation  : group17_logref_results.png")
    print(f"\n  Done.")

    # --- Compare Results ---
    print("\n" + "="*60)
    print("MODEL COMPARISON (RANDOM FOREST vs LOGISTIC REGRESSION)")
    print("="*60)
    print(f"{'Metric':<12} {'Random Forest':<15} {'Logistic Regression':<20}")
```

```python
    for metric in ["accuracy", "precision", "recall", "f1"]:
        print(f"{metric.capitalize():<12} {rf_metrics[metric]:<15.4f} {lr_metrics[metric]:<20.4f}")

if __name__ == "__main__":
    main()
```

```
============================================================
  GROUP PROJECT #17 - MORTGAGE APPROVAL PREDICTION
  GRAD 50400 | Spring 2026
============================================================

============================================================
  Loading 2024 HMDA data from: state_IN-KY-OH-WV_2024.csv
============================================================
  Raw shape: (976748, 99)
  Imputing 329,023 missing values in 'loan_to_value_ratio' with median=79.9770
  Imputing 350,902 missing values in 'interest_rate' with median=6.9900
  Imputing 130,468 missing values in 'income' with median=82.0000
  Imputing 214,460 missing values in 'property_value' with median=255000.0000
  Imputing 329,926 missing values in 'debt_to_income_ratio' with median=39.0000

  Class Balance (2024):
    Approved (1):    550,972  (56.4%)
    Denied   (0):    425,776  (43.6%)
  Final shape:  (976748, 8)

  Train set: 781,398 rows
  Test  set: 195,350 rows

============================================================
  SCALABILITY TEST [Random Forest]
============================================================
RF   10% (  78,139 rows) | train=  1.17s | infer/10k=0.0120s | acc=0.9603
RF   25% ( 195,349 rows) | train=  3.09s | infer/10k=0.0107s | acc=0.9601
RF   50% ( 390,699 rows) | train=  7.56s | infer/10k=0.0117s | acc=0.9613
RF  100% ( 781,398 rows) | train= 18.39s | infer/10k=0.0120s | acc=0.9603

  Scalability Summary [Random Forest] - (80/20 holdout split) :
Sample Size  N Train  Train Time (s)  Infer / 10k (s)  Accuracy
        10%    78139            1.17           0.0120    0.9603
        25%   195349            3.09           0.0107    0.9601
        50%   390699            7.56           0.0117    0.9613
       100%   781398           18.39           0.0120    0.9603


============================================================
  SCALABILITY TEST [Logistic Regression]
```

```
============================================================
LR  10% (  78,139 rows) | train=  0.10s | infer/10k=0.0007s | acc=0.5342
LR  25% ( 195,349 rows) | train=  0.30s | infer/10k=0.0006s | acc=0.5357
LR  50% ( 390,699 rows) | train=  0.68s | infer/10k=0.0007s | acc=0.5337
LR 100% ( 781,398 rows) | train=  1.51s | infer/10k=0.0006s | acc=0.5352

  Scalability Summary [Logistic Regression] - (80/20 holdout split) :
Sample Size  N Train  Train Time (s)  Infer / 10k (s)  Accuracy
        10%    78139            0.10           0.0007    0.5342
        25%   195349            0.30           0.0006    0.5357
        50%   390699            0.68           0.0007    0.5337
       100%   781398            1.51           0.0006    0.5352

============================================================
  HYPERPARAMETER TUNING - [Random Forest] - RandomizedSearchCV (3-fold CV)
============================================================
Fitting 3 folds for each of 10 candidates, totalling 30 fits

RF tuning completed in 1237.7s
RF Best CV F1: 0.9704
RF Best params: {'n_estimators': 100, 'min_samples_split': 5, 'min_samples_leaf': 2, 'max_features': 'sqrt', 'max_depth': 20,
'class_weight': None}

============================================================
 EVALUATION RESULTS [Random Forest] - Evaluate on 2024 holdout test set
============================================================
 Accuracy : 0.9666
 Precision : 0.9760
 Recall : 0.9645
 F1-Score : 0.9702

 Confusion Matrix: [Random Forest] - Evaluate on 2024 holdout test set
[[ 82547   2608]
 [  3916 106279]]

 Classification Report: [Random Forest] - Evaluate on 2024 holdout test set
              precision    recall  f1-score   support

      Denied       0.95      0.97      0.96     85155
    Approved       0.98      0.96      0.97    110195
```

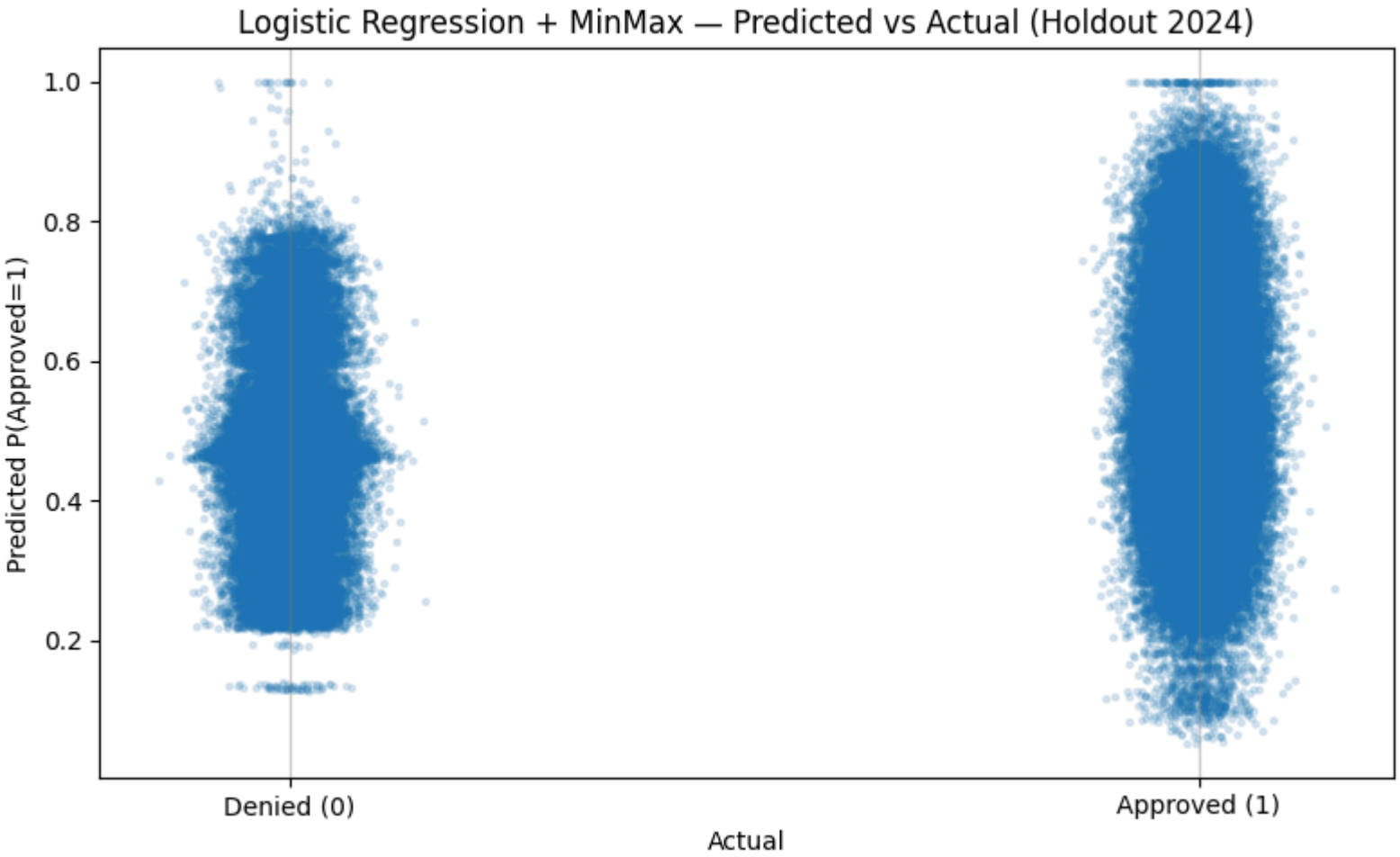
```
    accuracy                           0.97    195350
   macro avg       0.97      0.97      0.97    195350
weighted avg       0.97      0.97      0.97    195350

  SUCCESS CRITERIA CHECK: [Random Forest] - Evaluate on 2024 holdout test set
    Accuracy >= 0.90  --> PASS (0.9666)
    Precision >= 0.90 --> PASS (0.9760)
    Recall >= 0.90    --> PASS (0.9645)
    F1 >= 0.90        --> PASS (0.9702)
```



```
[Random Forest] - Inference time per 10k records (Evaluate on 2024 holdout test set): 0.0000s (target ≤ 1.0s)

============================================================
  SCALABILITY TEST [Random Forest]
============================================================
RF   10% (  78,139 rows) | train=  0.64s | infer/10k=0.0103s | acc=0.9603
RF   25% ( 195,349 rows) | train=  2.30s | infer/10k=0.0105s | acc=0.9601
RF   50% ( 390,699 rows) | train=  6.70s | infer/10k=0.0105s | acc=0.9613
RF  100% ( 781,398 rows) | train= 17.15s | infer/10k=0.0107s | acc=0.9603

============================================================
 5-FOLD CROSS-VALIDATION [Random Forest]
============================================================
 accuracy    : 0.9670 +/- 0.0004
 f1          : 0.9705 +/- 0.0003
 precision   : 0.9763 +/- 0.0003
 recall      : 0.9648 +/- 0.0006

============================================================
  HYPERPARAMETER TUNING - [LogReg + MinMaxScaler(0,1)] - GridSearchCV (3-fold CV)
============================================================
Fitting 3 folds for each of 48 candidates, totalling 144 fits

  LogReg tuning completed in 70.6s
  LogReg Best CV F1   : 0.6738
  LogReg Best params  : {'lr__C': 100, 'lr__class_weight': 'balanced', 'lr__max_iter': 1000, 'lr__penalty': 'l2', 'lr__solver':
'liblinear'}

  Top 15 hyperparameter attempts (by rank):
 rank_test_score  mean_test_score  std_test_score  mean_train_score  std_train_score
params
               1         0.673803        0.001106          0.673867         0.000310   {'lr__C': 100, 'lr__class_weight': 'bala
nced', 'lr__max_iter': 1000, 'lr__penalty': 'l2', 'lr__solver': 'liblinear'}
               1         0.673803        0.001106          0.673867         0.000310   {'lr__C': 100, 'lr__class_weight': 'bala
nced', 'lr__max_iter': 3000, 'lr__penalty': 'l2', 'lr__solver': 'liblinear'}
               3         0.673320        0.000778          0.673287         0.000365    {'lr__C': 10, 'lr__class_weight': 'bala
nced', 'lr__max_iter': 1000, 'lr__penalty': 'l2', 'lr__solver': 'liblinear'}
               3         0.673320        0.000778          0.673287         0.000365    {'lr__C': 10, 'lr__class_weight': 'bala
nced', 'lr__max_iter': 3000, 'lr__penalty': 'l2', 'lr__solver': 'liblinear'}
               5         0.672320        0.000757          0.672303         0.000327  {'lr__C': 0.01, 'lr__class_weight': 'bala
nced', 'lr__max_iter': 1000, 'lr__penalty': 'l2', 'lr__solver': 'liblinear'}
```

```
               5         0.672320        0.000757          0.672303         0.000327  {'lr__C': 0.01, 'lr__class_weight': 'bala
nced', 'lr__max_iter': 3000, 'lr__penalty': 'l2', 'lr__solver': 'liblinear'}
               7         0.672262        0.000954          0.672253         0.000235      {'lr__C': 0.01, 'lr__class_weight':
'balanced', 'lr__max_iter': 1000, 'lr__penalty': 'l2', 'lr__solver': 'lbfgs'}
               7         0.672262        0.000954          0.672253         0.000235      {'lr__C': 0.01, 'lr__class_weight':
'balanced', 'lr__max_iter': 3000, 'lr__penalty': 'l2', 'lr__solver': 'lbfgs'}
               9         0.671009        0.000915          0.670963         0.000243     {'lr__C': 0.001, 'lr__class_weight':
'balanced', 'lr__max_iter': 1000, 'lr__penalty': 'l2', 'lr__solver': 'lbfgs'}
               9         0.671009        0.000915          0.670963         0.000243     {'lr__C': 0.001, 'lr__class_weight':
'balanced', 'lr__max_iter': 3000, 'lr__penalty': 'l2', 'lr__solver': 'lbfgs'}
              11         0.670847        0.000958          0.670808         0.000142     {'lr__C': 1, 'lr__class_weight': 'bala
nced', 'lr__max_iter': 1000, 'lr__penalty': 'l2', 'lr__solver': 'liblinear'}
              11         0.670847        0.000958          0.670808         0.000142     {'lr__C': 1, 'lr__class_weight': 'bala
nced', 'lr__max_iter': 3000, 'lr__penalty': 'l2', 'lr__solver': 'liblinear'}
              13         0.670828        0.001113          0.670750         0.000109 {'lr__C': 0.001, 'lr__class_weight': 'bala
nced', 'lr__max_iter': 1000, 'lr__penalty': 'l2', 'lr__solver': 'liblinear'}
              13         0.670828        0.001113          0.670750         0.000109 {'lr__C': 0.001, 'lr__class_weight': 'bala
nced', 'lr__max_iter': 3000, 'lr__penalty': 'l2', 'lr__solver': 'liblinear'}
              15         0.670498        0.001028          0.670565         0.001778       {'lr__C': 0.1, 'lr__class_weight':
'balanced', 'lr__max_iter': 1000, 'lr__penalty': 'l2', 'lr__solver': 'lbfgs'}

============================================================
 EVALUATION RESULTS [Logistic Regression + MinMax] - Evaluate on 2024 holdout test set
============================================================
 Accuracy : 0.6810
 Precision : 0.7943
 Recall : 0.5863
 F1-Score : 0.6746

 Confusion Matrix: [Logistic Regression + MinMax] - Evaluate on 2024 holdout test set
[[68424 16731]
 [45589 64606]]

 Classification Report: [Logistic Regression + MinMax] - Evaluate on 2024 holdout test set
              precision    recall  f1-score   support

      Denied       0.60      0.80      0.69     85155
    Approved       0.79      0.59      0.67    110195

    accuracy                           0.68    195350
   macro avg       0.70      0.69      0.68    195350
```

```
weighted avg       0.71      0.68      0.68    195350

  SUCCESS CRITERIA CHECK: [Logistic Regression + MinMax] - Evaluate on 2024 holdout test set
    Accuracy >= 0.90  --> FAIL (0.6810)
    Precision >= 0.90 --> FAIL (0.7943)
    Recall >= 0.90    --> FAIL (0.5863)
    F1 >= 0.90        --> FAIL (0.6746)
```



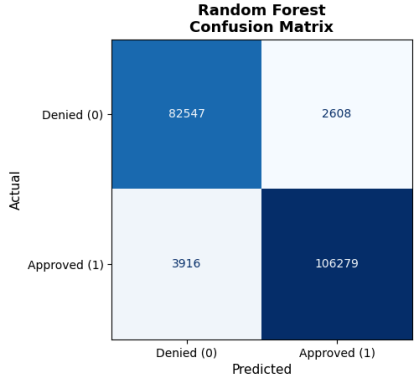
```
[Logistic Regression + MinMax] - Inference time per 10k records (Evaluate on 2024 holdout test set): 0.0019s (target ≤ 1.0s)

============================================================
 SCALABILITY TEST [Logistic Regression + MinMaxScaler(0,1)]
============================================================
LR+MM  10% (  78,139 rows) | train=  0.12s | infer/10k=0.0017s | acc=0.5232
LR+MM  25% ( 195,349 rows) | train=  0.21s | infer/10k=0.0018s | acc=0.5218
LR+MM  50% ( 390,699 rows) | train=  0.44s | infer/10k=0.0015s | acc=0.5209
LR+MM 100% ( 781,398 rows) | train=  0.87s | infer/10k=0.0016s | acc=0.5228

============================================================
 5-FOLD CROSS-VALIDATION [Logistic Regression + MinMax]
============================================================
 accuracy    : 0.6797 +/- 0.0017
 f1          : 0.6737 +/- 0.0019
 precision   : 0.7920 +/- 0.0023
 recall      : 0.5861 +/- 0.0025

============================================================
  OUT-OF-TIME VALIDATION (Train=2023, Test=2024) - [Random Forest]
============================================================

============================================================
  Loading 2023 HMDA data from: state_IN-KY-OH-WV_2023.csv
============================================================
  Raw shape: (894865, 99)
  Imputing 291,423 missing values in 'loan_to_value_ratio' with median=80.0000
  Imputing 329,065 missing values in 'interest_rate' with median=6.9900
  Imputing 95,531 missing values in 'income' with median=80.0000
  Imputing 193,659 missing values in 'property_value' with median=245000.0000
  Imputing 279,284 missing values in 'debt_to_income_ratio' with median=38.0000

  Class Balance (2023):
    Approved (1):    506,319  (56.6%)
    Denied   (0):    388,546  (43.4%)
  Final shape:  (894865, 8)

============================================================
 EVALUATION RESULTS [Out-of-Time for Random Forest: Train 2023 --> Test 2024]
============================================================
 Accuracy : 0.8581
```

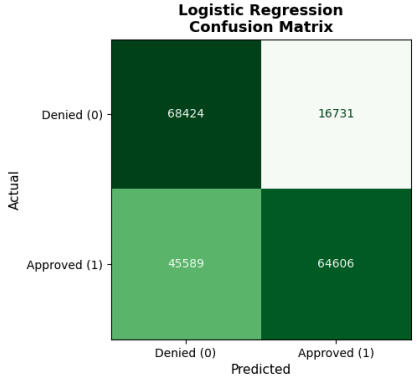
```
 Precision : 0.8409
 Recall : 0.9230
 F1-Score : 0.8801

 Confusion Matrix: [Out-of-Time for Random Forest: Train 2023 --> Test 2024]
[[329586  96190]
 [ 42417 508555]]

 Classification Report: [Out-of-Time for Random Forest: Train 2023 --> Test 2024]
              precision    recall  f1-score   support

      Denied       0.89      0.77      0.83    425776
    Approved       0.84      0.92      0.88    550972

    accuracy                           0.86    976748
   macro avg       0.86      0.85      0.85    976748
weighted avg       0.86      0.86      0.86    976748

  SUCCESS CRITERIA CHECK: [Out-of-Time for Random Forest: Train 2023 --> Test 2024]
    Accuracy >= 0.90  --> FAIL (0.8581)
    Precision >= 0.90 --> FAIL (0.8409)
    Recall >= 0.90    --> PASS (0.9230)
    F1 >= 0.90        --> FAIL (0.8801)

============================================================
  OUT-OF-TIME VALIDATION (Train=2023, Test=2024) - [Logistic Regression + MinMax]
============================================================

============================================================
  Loading 2023 HMDA data from: state_IN-KY-OH-WV_2023.csv
============================================================
  Raw shape: (894865, 99)
  Imputing 291,423 missing values in 'loan_to_value_ratio' with median=80.0000
  Imputing 329,065 missing values in 'interest_rate' with median=6.9900
  Imputing 95,531 missing values in 'income' with median=80.0000
  Imputing 193,659 missing values in 'property_value' with median=245000.0000
  Imputing 279,284 missing values in 'debt_to_income_ratio' with median=38.0000

  Class Balance (2023):
    Approved (1):    506,319  (56.6%)
    Denied   (0):    388,546  (43.4%)
```

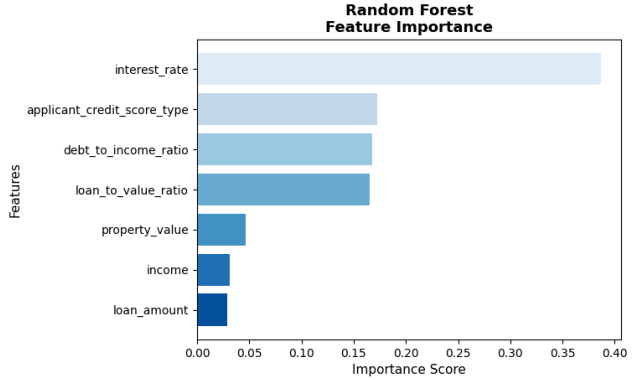
```
  Final shape:  (894865, 8)

============================================================
 EVALUATION RESULTS [Out-of-Time for Logistic Regression + MinMax: Train 2023 --> Test 2024]
============================================================
 Accuracy : 0.4947
 Precision : 0.5513
 Recall : 0.5597
 F1-Score : 0.5555

 Confusion Matrix: [Out-of-Time for Logistic Regression + MinMax: Train 2023 --> Test 2024]
[[174830 250946]
 [242597 308375]]

 Classification Report: [Out-of-Time for Logistic Regression + MinMax: Train 2023 --> Test 2024]
              precision    recall  f1-score   support

      Denied       0.42      0.41      0.41    425776
    Approved       0.55      0.56      0.56    550972

    accuracy                           0.49    976748
   macro avg       0.49      0.49      0.49    976748
weighted avg       0.49      0.49      0.49    976748

  SUCCESS CRITERIA CHECK: [Out-of-Time for Logistic Regression + MinMax: Train 2023 --> Test 2024]
    Accuracy >= 0.90  --> FAIL (0.4947)
    Precision >= 0.90 --> FAIL (0.5513)
    Recall >= 0.90    --> FAIL (0.5597)
    F1 >= 0.90        --> FAIL (0.5555)

============================================================
  METRICS COMPARISON: 80/20 HOLDOUT vs OUT-OF-TIME (Train 2023 -> Test 2024)
============================================================
              Model  80/20 Accuracy  80/20 Precision  80/20 Recall  80/20 F1  2023->2024 Accuracy  2023->2024 Precision  2023->
2024 Recall  2023->2024 F1
      Random Forest        0.966604         0.976049      0.964463  0.970221             0.858093              0.840941
0.923014       0.880068
Logistic Regression        0.680983         0.794300      0.586288  0.674624             0.494708              0.551338
0.559693       0.555484

[Random Forest] - Feature Importance :
```

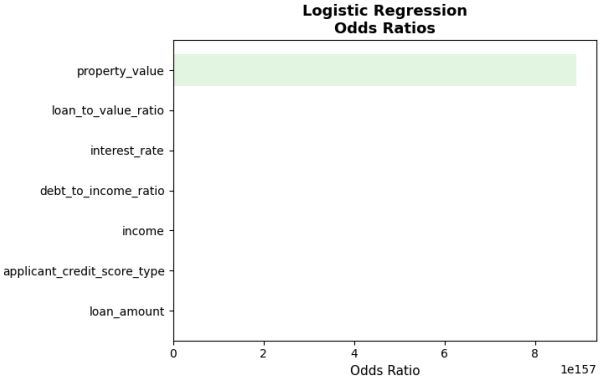
```
                       Feature  Importance Score
0                interest_rate          0.386471
1  applicant_credit_score_type          0.172783
2         debt_to_income_ratio          0.167836
3          loan_to_value_ratio          0.165415
4               property_value          0.046946
5                       income          0.031292
6                  loan_amount          0.029257

[Logistic Regression] - coefficient/odds-ratio interpretation (sorted by |coefficient|):
                       Feature  Coefficient     Odds Ratio
6               property_value   363.693401  8.913272e+157
1          loan_to_value_ratio    -5.637765   3.560817e-03
2                interest_rate     4.328633   7.584051e+01
4         debt_to_income_ratio    -1.696457   1.833320e-01
3                       income     0.866694   2.379033e+00
5  applicant_credit_score_type    -0.589619   5.545383e-01
0                  loan_amount    -0.541271   5.820080e-01

 Visualisation saved --> rf_lr_results.png
```

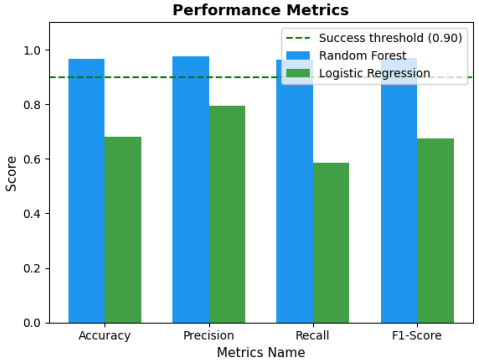
Group 17 — Random Forest: Mortgage Approval Prediction
HMDA 2024 | OH, WV, IN, KY

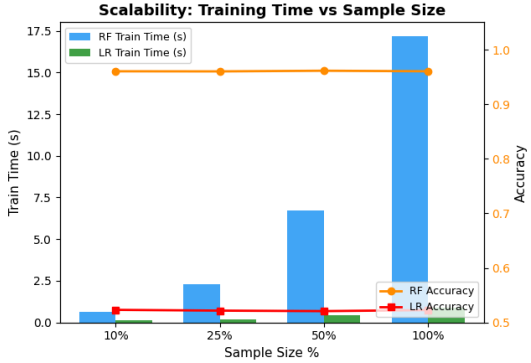












```
============================================================
  FINAL MODEL SUMMARY [RANDOM FOREST]
============================================================
  Model          : RandomForestClassifier (scikit-learn)
  Best params    : {'n_estimators': 100, 'min_samples_split': 5, 'min_samples_leaf': 2, 'max_features': 'sqrt', 'max_depth': 2
0, 'class_weight': None}
  Features used  : ['loan_amount', 'loan_to_value_ratio', 'interest_rate', 'income', 'debt_to_income_ratio', 'applicant_credit_
score_type', 'property_value']
  Train rows     : 781,398
  Test rows      : 195,350
  Accuracy       : 0.9666
  Precision      : 0.9760
  Recall         : 0.9645
  F1-Score       : 0.9702

  Visualisation  : group17_rf_results.png

  Done.

============================================================
  FINAL MODEL SUMMARY [LOGISTIC REGRESSION]
============================================================
  Model          : LogisticRegression (scikit-learn)
  Best params    : {'lr__C': 100, 'lr__class_weight': 'balanced', 'lr__max_iter': 1000, 'lr__penalty': 'l2', 'lr__solver': 'lib
linear'}
  Features used  : ['loan_amount', 'loan_to_value_ratio', 'interest_rate', 'income', 'debt_to_income_ratio', 'applicant_credit_
score_type', 'property_value']
  Train rows     : 781,398
  Test rows      : 195,350
  Accuracy       : 0.6810
  Precision      : 0.7943
  Recall         : 0.5863
  F1-Score       : 0.6746

  Visualisation  : group17_logref_results.png

  Done.

============================================================
MODEL COMPARISON (RANDOM FOREST vs LOGISTIC REGRESSION)
============================================================
```

```
Metric       Random Forest   Logistic Regression
Accuracy     0.9666          0.6810
Precision    0.9760          0.7943
Recall       0.9645          0.5863
F1           0.9702          0.6746
```

In [ ]: